University of Science and Technology of Hanoi (USTH)
Department of Information and Communication Technology

Introduction to Cryptography — Mid-term Project Report

Transaction-Bound Authentication for Online Banking: Implementation and Security Evaluation

Group N

Full name	**Student ID**	**Role**
MEMBER 1 FULL NAME	STUDENT ID	Leader
MEMBER 2 FULL NAME	STUDENT ID	Member
MEMBER 3 FULL NAME	STUDENT ID	Member
MEMBER 4 FULL NAME	STUDENT ID	Member
MEMBER 5 FULL NAME	STUDENT ID	Member
MEMBER 6 FULL NAME	STUDENT ID	Member

Lecturer: LECTURER FULL NAME
16 November 2026

Abstract

Online banking runs on the customer's own computer, which malware can fully control. A man-in-the-browser (MitB) Trojan, or an equivalent real-time phishing proxy, can silently change the recipient or the amount of a transfer after the user has typed it, while the screen still shows the intended details. A one-time password that is not tied to the transaction does not stop this attack, because the stolen code authorises whatever transaction the server holds.

This project implements a small but complete demo bank and four interchangeable transaction-authentication modes on top of it: `baseline` (a plain time-based OTP), `hmac` (an 8-digit code computed over the exact transaction with HMAC-SHA256, in the style of OCRA), `sig` (an ECDSA P-256 signature over the exact transaction), and `sig_blind` (the same signature, but the phone shows the user nothing). Transaction binding is not a new idea; the contribution of this report is a *controlled comparison* of the four modes on one code base, with the blind-signing experiment isolating the role of the trusted display from the role of the cryptography.

We evaluate the four modes against a threat model of seven attacker capabilities and seven security goals, using an automated harness that drives a virtual phone and a simulated MitB across thirteen scenarios in all four modes (56 cells), plus a live MitB demonstration through `mitmproxy`, an online-guessing experiment, and micro-benchmarks. Every result in the report is produced by the project's `make eval` target. The measured matrix confirms that transaction binding (`hmac`, `sig`) blocks recipient and amount tampering that `baseline` does not, that asymmetric signatures additionally survive a server database leak and yield third-party–verifiable evidence, and that `sig_blind` — strong cryptography with no trusted display — fails exactly the attacks an attentive user would otherwise catch. The practical cost is small: an iPhone signs a transaction in well under a millisecond. We also state clearly where the guarantees stop: the per-transaction guessing limit, the gap between "verifiable evidence" and non-repudiation, and the idealised attentive-user model.

Contents

1 Introduction **3**
- 1.1 Problem . . . 3
- 1.2 Approach . . . 3
- 1.3 Research questions . . . 3
- 1.4 Contributions . . . 3

2 Background and Related Work **4**
- 2.1 Man-in-the-browser and real-time phishing . . . 4
- 2.2 Transaction binding and the trusted display . . . 4
- 2.3 Do users actually check? . . . 4
- 2.4 Comparison with existing solutions . . . 5

3 Threat Model and Security Goals **5**
- 3.1 Trust assumptions . . . 5
- 3.2 Attacker capabilities . . . 5
- 3.3 Security goals . . . 6

4 System Design **7**
- 4.1 Components and trust boundaries . . . 7
- 4.2 Transaction encoding . . . 7
- 4.3 The four modes . . . 8
- 4.4 Device enrolment . . . 8
- 4.5 Confirmation under a man-in-the-browser . . . 8

5 Implementation **9**
5.1 Canonical encoding . . . 9
5.2 Signing on the phone . . . 9
5.3 Server-side verification order . . . 10
5.4 The authenticator interface . . . 10

6 Security Evaluation **11**
6.1 Methodology . . . 11
6.2 Case study: man-in-the-browser . . . 11
6.3 Scenario results . . . 11
6.4 Online guessing . . . 12
6.5 Server compromise and verifiable evidence . . . 13
6.6 Summary against security goals . . . 13

7 Performance **13**
7.1 P1: operation latency on the host . . . 14
7.2 P2: signing on a real phone . . . 14
7.3 P3: data sizes . . . 15

8 Discussion and Limitations **15**
8.1 What each layer buys . . . 15
8.2 Limitations . . . 16
8.3 Positioning . . . 16

9 Conclusion **16**

A Reproducing the results **18**

B Live man-in-the-browser demo **18**

C Demo accounts **18**

D Contribution table **18**

1 Introduction

1.1 Problem

A bank customer authenticates to the bank, but the device that carries the customer's intent to the bank — the web browser on a personal computer — is not trustworthy. Malware in the browser, known as a man-in-the-browser (MitB) Trojan, can read and rewrite every request and response. When the customer asks to send money to account A, the Trojan can change the request to account B while leaving the page on screen showing account A. The same manipulation is available to a real-time phishing proxy that relays the victim's traffic through an attacker-controlled site. A one-time password (OTP) delivered out of band, such as a time-based code from an authenticator app, does *not* defend against this: the code proves that the right person is present, but says nothing about *which* transaction is being approved, so the attacker simply uses it to authorise the tampered transfer.

1.2 Approach

The defence studied here is *transaction binding*: the code or signature that the customer returns is computed over the exact transaction, so a code that was produced for "send 1,000,000 to account A" cannot authorise "send 9,000,000 to account B". For this to help a human, the transaction must be shown to the customer on a second, trusted device (the phone) — the "what-you-see-is-what-you-sign" (WYSIWYS) principle — so the customer can notice a mismatch before approving.

We build a demo bank with four interchangeable modes and compare them under one threat model:

- `baseline` — a plain TOTP, *not* bound to the transaction;
- `hmac` — an 8-digit code over the exact transaction (HMAC-SHA256, OCRA-style);
- `sig` — an ECDSA P-256 signature over the exact transaction;
- `sig_blind` — identical cryptography to `sig`, but the phone displays no transaction details.

The `sig_blind` mode is a deliberate experiment: it keeps the cryptography of `sig` but removes the trusted display, so that the difference between `sig` and `sig_blind` measures the role of the display alone.

1.3 Research questions

RQ1. Does binding the authentication code to the transaction stop MitB and real-time phishing better than an ordinary OTP?

RQ2. How do symmetric binding (HMAC) and asymmetric binding (signatures) differ in security and in cost?

RQ3. What does the trusted display (WYSIWYS) contribute, independently of the cryptography? The blind-signing mode answers this.

1.4 Contributions

1. A complete, reproducible demo: a FastAPI bank, a browser-based phone authenticator, a shared cryptographic core, and four modes selectable at run time (sections 4 and 5).
2. A threat model of seven attacker capabilities and seven security goals, and an automated harness that checks thirteen scenarios in all four modes (56 cells) against an expected matrix, so that any regression is caught mechanically (section 6).

3. A controlled measurement of what each layer buys: transaction binding, asymmetric keys, and the trusted display, including a live MitB demonstration and micro-benchmarks (sections 6 and 7).

4. An explicit account of the limits: the per-transaction guessing bound, the gap between verifiable evidence and non-repudiation, and the idealised user model (section 8).

We do *not* claim transaction binding as a novel mechanism; it is long established (section 2). The contribution is the controlled comparison on one system.

2 Background and Related Work

2.1 Man-in-the-browser and real-time phishing

A man-in-the-browser Trojan sits inside the victim's browser and intercepts the calls between the user interface and the network, so it sees the transaction the user intends and can substitute a different one before it reaches the bank, while keeping the displayed page unchanged. A real-time phishing proxy achieves the same position from the network side: the victim interacts with an attacker-controlled copy of the bank site, and the attacker relays and rewrites the traffic. Because both end up between the user's intent and the bank's record of the transaction, we treat them as one capability in our threat model (section 3).

2.2 Transaction binding and the trusted display

The idea of binding the authorisation to the specific transaction, and of showing the transaction on a trusted device so the user signs what they see (WYSIWYS), is well established in both research and deployment. Hiltgen, Kramp and Weigold surveyed secure internet-banking authentication and argued for transaction-specific confirmation [1]; the Zurich Trusted Information Channel (ZTIC) realised a trusted device that displays and confirms transaction data [2]. In deployment, chipTAN and photoTAN schemes derive a transaction code from the transaction data, and the challenge-response standard OCRA includes a signature/transaction mode (RFC 6287, signature mode) [3]. Regulation has made the principle mandatory for European payment service providers: the "dynamic linking" requirement of Commission Delegated Regulation (EU) 2018/389, Article 5, requires the authentication code to be specific to the amount and the payee [4]. More recent web standards, such as W3C Secure Payment Confirmation, carry transaction details into the signed authenticator assertion [5].[1]

Against this background, the four modes in this report are not new designs. `baseline` is RFC 4226/6238 HOTP/TOTP [6, 7]; `hmac` is an OCRA-*style* code (it uses HMAC-SHA256 over a canonical transaction with a domain-separation tag, but it is *not* standards-exact OCRA: there is no OCRASuite string and no RFC 6287 `DataInput` layout); `sig`/`sig_blind` apply an ECDSA P-256 signature (FIPS 186-5) over the same canonical transaction. The contribution is the controlled comparison and the blind-signing experiment, not the mechanisms.

2.3 Do users actually check?

WYSIWYS only helps if the user reads the trusted display and notices a mismatch. This assumption is known to be optimistic. AlZomai, AlFayyadh, Jøsang and McCullagh ran a simulated-bank user study of transaction authorisation [8], and Haupert and Gabert surveyed real banking customers on where they look for the signed transaction details [9]. We use these works to bound the interpretation of our attentive-user results (sections 6 and 8); we do not

[1] W3C Secure Payment Confirmation is, at the time of writing, a Candidate Recommendation Draft, not a Recommendation; cite its exact status when used.

reproduce their specific detection rates here, because those must be read from the original papers before quoting.[2]

2.4 Comparison with existing solutions

Table 1 positions the four implemented modes against representative existing approaches on the properties that matter for MitB defence. It is a qualitative map, not a claim of novelty.

Table 1: Where the four modes sit among transaction-authentication approaches.

Approach	Binds code to transaction?	Trusted display	Asymmetric	Standards
Plain OTP / TOTP (`baseline`)	No	No	No	RFC 4226/6238
chipTAN / photoTAN	Yes	Yes	No	vendor
OCRA signature mode	Yes	Yes	No	RFC 6287
HMAC transaction code (`hmac`)	Yes	Yes	No	OCRA-style
Signature (`sig`)	Yes	Yes	Yes	FIPS 186-5
Signature, no display (`sig_blind`)	Yes	No	Yes	FIPS 186-5
WebAuthn / W3C SPC	Yes (SPC)	Yes	Yes	W3C

3 Threat Model and Security Goals

3.1 Trust assumptions

We assume that TLS is secure; that the bank server is honest but its database may leak (read-only exposure); that the phone running the authenticator is trusted; and that the environment is clean at the moment of device enrolment (trust on first use). These assumptions are discussed, and their fragility noted, in section 8.

3.2 Attacker capabilities

Table 2 lists the attacker capabilities in scope. The scenarios in the last column (section 6) exercise each capability.

Table 2: Attacker capabilities.

ID	Attacker	Capability	Scenarios
A1	Man-in-the-browser	Reads and rewrites everything in the browser	S2a, S2b, S3, S4
A2	Real-time phishing proxy	Relays and rewrites traffic in real time	S2a, S2b, S3, S4 (see note)
A3	Replay	Re-uses an old code or signature	S6, S7, S8
A4	Online guessing	Guesses codes against the live server	S9
A5	Database theft	Holds a copy of the server database	S10
A6	Malware on the phone	Compromises the phone, or PC and phone together	out of scope
A7	Social engineering	Persuades the user to approve	out of scope

[2] A preliminary review obtained only press-release figures (reported as 21% of obvious attacks and 61% of subtle attacks succeeding); these are not cited as facts and must be checked against the original publications before inclusion.

Note on A2 (equivalence to A1). We give the phishing proxy no separate scenario because, in this model, it can do exactly what the MitB layer does: stand between the user and the bank, rewrite the transfer request, and rewrite the returned page (including the QR code). The only difference is how the attacker reaches that position (a fake site versus malware), which does not change the outcome. Every claim about real-time phishing in this report (RQ1) therefore refers to scenarios S2a/S2b/S3/S4 and relies on this equivalence; we did *not* run a separate phishing experiment.

Note on A6 (phone compromise, out of scope). The assumption that the phone is trusted and separate from the PC is a strong one, and the literature shows it is easily lost: cross-infection of PC and phone [10], cross-platform synchronisation that erases the gap between the two devices [11], and the bank app and authenticator running on the same phone [12]. We treat A6 as out of scope and return to it in section 8.

3.3 Security goals

Table 3 states the goals the modes are measured against. Two goals carry limitations that are important enough to state up front.

Table 3: Security goals and the scenarios that test them.

ID	Goal	Meaning	Tested by
G1	Authorization integrity	Server executes T only if the device produced a code for exactly T	S4, S5
G2	Freshness	No replay, no late use	S6, S7
G3	Single use	One code, one execution, even under concurrency	S8
G4	Guessing limit	Bounded guessing *per transaction* (see limitation)	S9
G5	DB-leak resilience	A leaked database does not let the attacker forge transfers	S10
G6	Verifiable evidence	A third party can check the evidence (necessary, not sufficient, for non-repudiation)	S11
G7	WYSIWYS	The user saw exactly T (idealised attentive user)	S2a, S3

Limitation of G6 (verifiable evidence, not non-repudiation). Scenario S11 shows only that the evidence stored in the database can be checked with a public key, without any secret of the bank's. It is *not* non-repudiation, for two reasons. First, the authenticator is a web application served by the bank itself, so the bank could serve different JavaScript that uses the signing key for an arbitrary transaction; a non-extractable key prevents *exporting* the key, not *using* it. Second, the public key lives only in the bank's `devices` table, with no independent party attesting that it belongs to the user, so the bank could substitute a key. We therefore write *verifiable evidence*, not *non-repudiation*, for S11, and mark the `sig_blind` cell with an asterisk (the signature proves the device signed, not that the user saw the transaction).

Limitation of G4 (per-transaction, not per-user). The three-attempt limit is enforced per transaction (`server/verify.py`, `settings.max_attempts`); there is no per-user or per-device counter, and transaction creation is not rate-limited. An attacker with the password but no phone (A4) can create k transactions and guess three times each, so the cumulative success probability is about $k \cdot p$. This limitation is quantified in section 6.

4 System Design

4.1 Components and trust boundaries

Figure 1 shows the components and which side of the trust boundary each one is on. The victim's PC (browser, and the MitB) is untrusted; the phone authenticator and the bank server are trusted. The cryptographic core (`tba_core`) is shared by the server and, through cross-checked test vectors, by the phone; the evaluation harness drives the same core and the server in process.

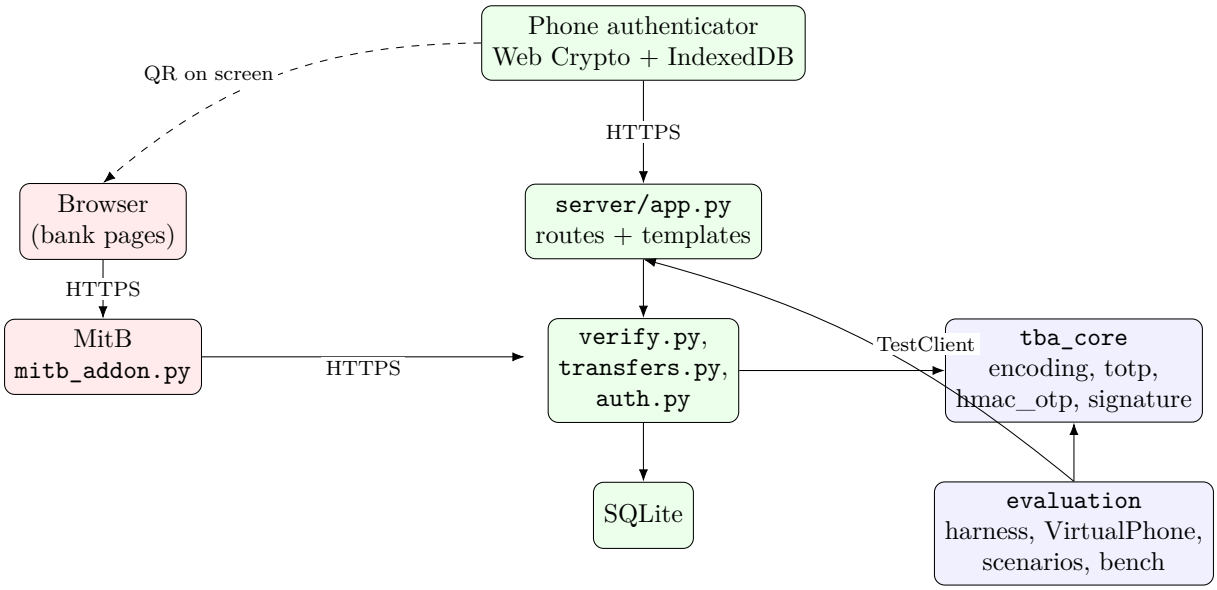


Figure 1: Architecture and trust boundaries (F1). Green: trusted; red: untrusted; blue: test/verification code. The QR code is the only channel from the untrusted browser to the trusted phone, and the phone talks to the server on its own HTTPS channel.

4.2 Transaction encoding

A transaction is a fixed JSON object; every field is validated identically in Python and in JavaScript (table 4). The *canonical* form is `json.dumps(tx, sort_keys=True, separators=(",",":"), ensure_ascii=True)` encoded as ASCII. Parsing is strict: duplicate keys, floating-point numbers, `NaN`/`Infinity`, non-ASCII bytes, and any string that does not re-serialise to the identical bytes are rejected. The transaction QR payload is `"tba1:"` followed by the base64url of the canonical bytes; the enrolment QR payload is `"tbaenroll1:"` followed by a token. The server always verifies against the canonical transaction stored in *its own* database, never a transaction sent by the client.

Table 4: Transaction fields and their constraints.

Field	Constraint
`v`	integer, equal to 1
`tx_id`	lower-case UUID `[0-9a-f]{8}-...-[0-9a-f]{12}`
`from`, `to`	`[0-9]{6,20}`
`to_name`	`[A-Z0-9 ]{1,64}`
`amount`	integer (not bool), $1 \leq$ `amount` $\leq 2^{53} - 1$
`currency`	`[A-Z]{3}`
`nonce`	16 random bytes, base64url without padding, exactly 22 chars
`exp`	integer, equal to creation time + 120 seconds

4.3 The four modes

Table 5 gives, for each mode, what the phone computes and what the server checks. Each device has its own K_{totp} (20 bytes) and K_{hmac} (32 bytes); the ECDSA key pair is generated on the phone. The domain-separation tags (`"TBA-HMAC-v1"`, `"TBA-SIG-v1"`), joined to the canonical bytes with a `0x00` byte, stop a code or signature for one purpose being reused for another.

Table 5: Cryptography of each mode (T4).

Mode	Phone computes	Server checks
`baseline`	HOTP(K_{totp}, $\lfloor t/30 \rfloor$), HMAC-SHA1, 6 digits	window ±1 step; reject step ≤ last used step
`hmac`	DT(HMAC-SHA256(K_{hmac}, tag ‖ `0x00` ‖ canon(*txn*))), 8 digits	recompute, `compare_digest`
`sig`	ECDSA-P256-SHA256(sk, tag ‖ `0x00` ‖ canon(*txn*)), raw 64 bytes	convert to DER, verify with public key
`sig_blind`	same as `sig`	same as `sig`

4.4 Device enrolment

Enrolment (fig. 2) establishes one device per user. Crucially, once a device exists the server *refuses* to enrol another from a web session alone: `GET /enroll` returns 409 and issues no token. Otherwise an attacker with only a session (A1, or a stolen password) could enrol their own key and approve every transaction, in every mode. In the demo, re-enrolment requires wiping the database (`make reset`); a real system needs an out-of-band process (section 8).

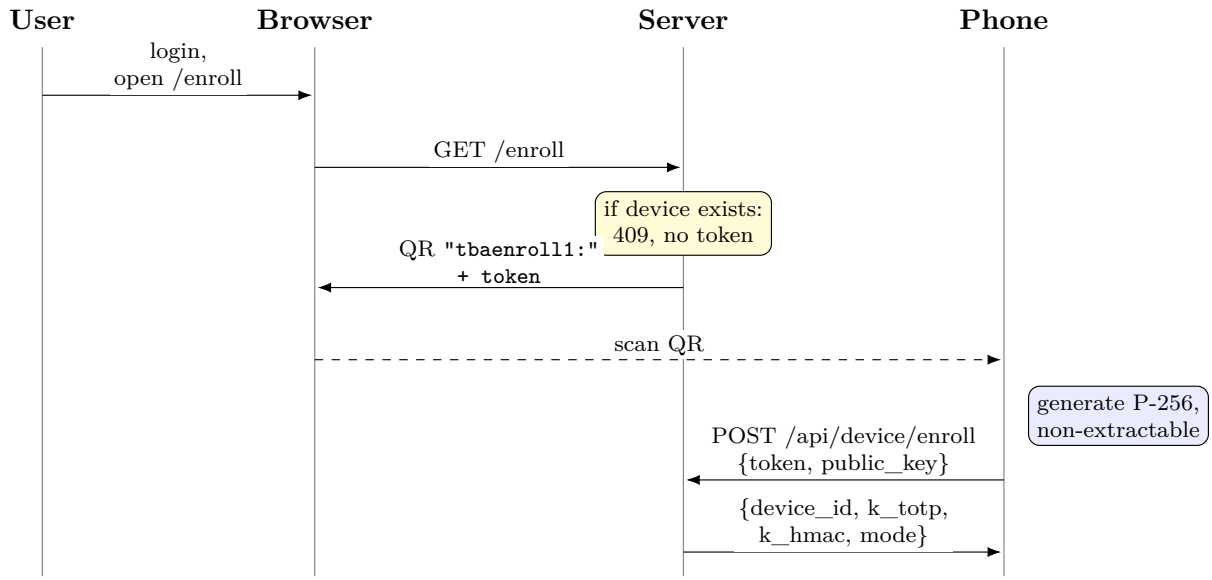


Figure 2: Device enrolment (F2). The HMAC keys are sent in the clear over HTTPS and stored by the phone as non-extractable keys in IndexedDB.

4.5 Confirmation under a man-in-the-browser

Figure 3 shows what happens to a transfer when the browser is compromised. The MitB rewrites the recipient to the attacker before the request reaches the server, and masks the returned confirmation page so the browser still shows the intended recipient. The transaction QR, however, encodes the transaction the server actually holds, so the phone displays the *true* recipient. In

`hmac` and `sig` an attentive user sees the mismatch and rejects; in `sig_blind` the phone shows nothing, so the user cannot notice. This is the mechanism measured in section 6 and shown in the screenshots of fig. 5.

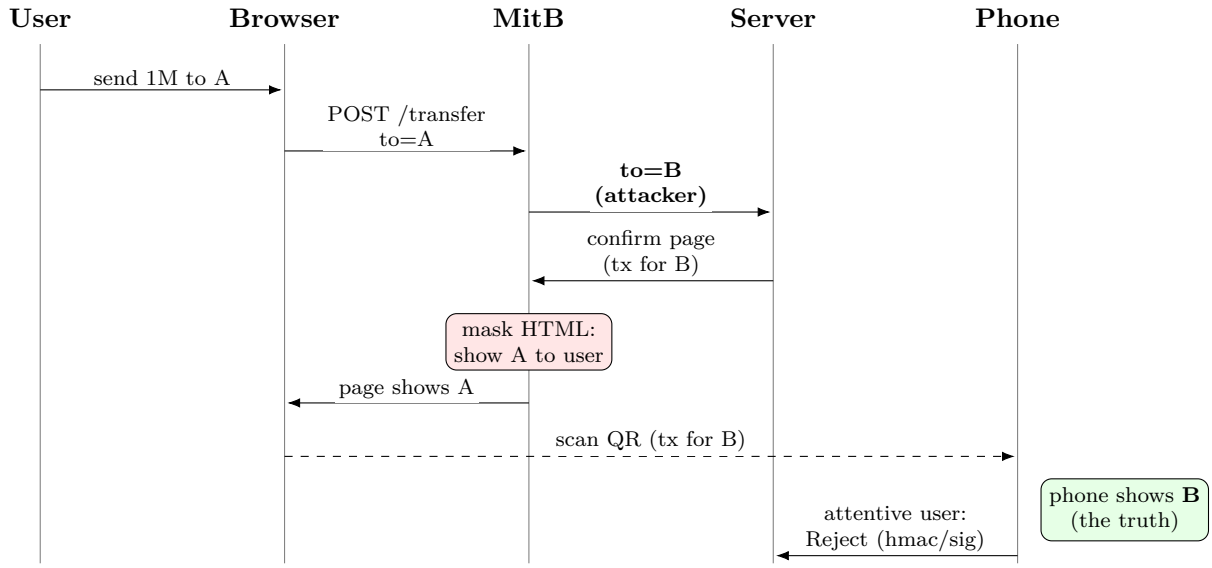


Figure 3: Confirmation under MitB (F3). The browser is deceived, but the phone reads the transaction the server holds, so `hmac`/`sig` expose the swap. In `sig_blind` the phone shows nothing and the user approves.

5 Implementation

The system is a FastAPI server with an SQLite database and Jinja2 templates (`server/`), a browser-based phone authenticator built from plain ES modules (`authenticator/`), and a shared, I/O-free cryptographic core (`tba_core/`). The server and the phone never share code directly; their agreement is enforced by a common set of test vectors (`shared/vectors.json`) that both a Python test and a Node test check.

5.1 Canonical encoding

The security of binding rests on both sides hashing *exactly* the same bytes. The canonical form fixes key order, removes whitespace, forbids non-ASCII, and is validated before use:

```
def canonical_bytes(tx: dict) -> bytes:
    validate_tx(tx)
    return json.dumps(tx, sort_keys=True,
                      separators=(",", ":"),
                      ensure_ascii=True).encode("ascii")
```

The matching JavaScript produces byte-identical output, which the cross-language vectors confirm (20 vectors, all passing).

5.2 Signing on the phone

The ECDSA key pair is generated in the browser with Web Crypto and is non-extractable; the signature is taken over the tagged, canonical transaction:

```
export function generateSigningKey() {
  // extractable=false applies to the private key.
```

```
  return crypto.subtle.generateKey(
    { name: "ECDSA", namedCurve: "P-256" }, false, ["sign", "verify"]);
}
export async function signTx(privateKey, tx) {
  return new Uint8Array(await crypto.subtle.sign(
    { name: "ECDSA", hash: "SHA-256" },
    privateKey, taggedMessage("TBA-SIG-v1", tx)));
}
```

A non-extractable key cannot be exported through the Web Crypto API; this is an API-level protection, not a hardware one, and it does not prevent the key being *used* (see the G6 limitation in section 3).

5.3 Server-side verification order

Each confirmation runs inside one `BEGIN IMMEDIATE` transaction (`write_tx`) so that concurrent submissions of the same code cannot both succeed (goal G3). The order is: (1) the `tx_id` exists and belongs to this user; (2) its state is `pending`; (3) it has not expired, else it becomes `expired`; (4) the transaction is rebuilt from the canonical bytes in the database; (5) the OTP or signature is verified — on failure `attempts` is incremented and, at three, the transaction becomes `failed`; (6) on success the balances move, the state becomes `executed`, and the OTP or signature is stored as evidence; (7) an audit record is written. Evidence of the wrong kind (an OTP in a signature mode, or a signature in an OTP mode) is rejected before step 1 and does *not* consume an attempt. The server verifies only the canonical transaction it stored, never one supplied by the client.

5.4 The authenticator interface

Figure 4 shows the authenticator's transaction screen. In `hmac` and `sig` it shows the amount, the recipient account and the recipient name read from the scanned QR; the user compares these with their intent before approving. (The figure is taken from the man-in-the-browser demo of section 6, so the recipient it shows is the attacker's.)

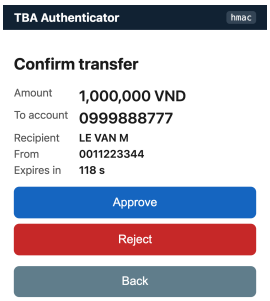


Figure 4: The authenticator's confirmation screen in `hmac` mode (F7).

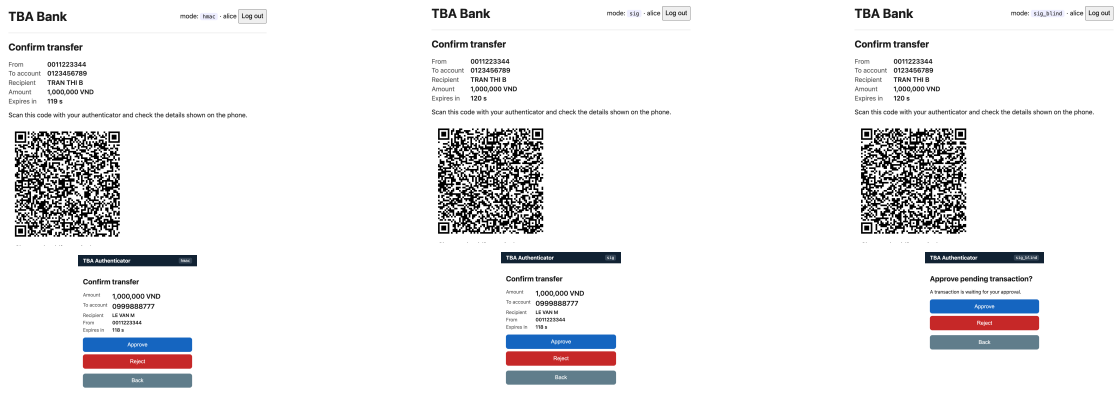
6 Security Evaluation

6.1 Methodology

The evaluation is automated. A `Lab` builds a fresh in-process server (with a controllable clock) for each run; a `VirtualPhone` plays the authenticator using `tba_core`; and a `Mitb` object rewrites requests and QR codes exactly as the live `mitmproxy` addon does. Two user models bound the human: an *attentive* user approves only when the phone's display matches their intent (and always approves when the phone shows nothing, as in blind signing), while a *careless* user always approves. Thirteen scenarios (S1–S13) are run in all four modes and each outcome is compared with a pre-declared expected value; `make scenarios` exits non-zero on any mismatch. All results below are the committed output of `make eval` and live in `results/`.

6.2 Case study: man-in-the-browser

Figure 5 shows the live attack through `mitmproxy`, for a transfer the user meant to send to **TRAN THI B (0123456789)**. The malware rewrites the recipient to **LE VAN M (0999888777)**, the attacker's account, and masks the bank's confirmation page so the laptop still shows TRAN THI B. The phone, which reads the transaction the server actually holds, tells the truth: in `hmac` and `sig` it shows LE VAN M, so an attentive user rejects; in `sig_blind` it shows no details, so the user approves the theft.



`hmac` `sig` `sig_blind`

Figure 5: Live man-in-the-browser (F4). Top row: the laptop confirmation page, deceived into showing TRAN THI B in every mode. Bottom row: the phone. In `hmac`/`sig` the phone shows the real recipient (0999888777, LE VAN M); in `sig_blind` it shows no details at all.

6.3 Scenario results

Table 6 is the full matrix (generated by `make scenarios`; ✓ = secure outcome, ✗ = insecure outcome, - = not applicable). The measured outcomes equal the expected outcomes in all 56 cells.

The matrix answers the research questions directly. Binding the code to the transaction (`hmac`, `sig`) blocks the recipient and amount swaps (S2a, S3) that `baseline` lets through, and both block tampering with any field after the code is issued (S4, S5) — this is RQ1. The difference between `hmac` and `sig` appears in S10 and S11 (RQ2, section 6.5). The difference between `sig` and `sig_blind` appears in S2a and S3 (RQ3): identical cryptography, but without the trusted

Table 6: Scenario outcomes across the four modes (T5). Source: `results/scenarios.csv`.

ID	Scenario	baseline	hmac	sig	sig_blind
S1	Honest transfer	✓	✓	✓	✓
S2a	MitB swaps recipient, attentive user	✗	✓	✓	✗
S2b	MitB swaps recipient, careless user	✗	✗	✗	✗
S3	MitB changes amount	✗	✓	✓	✗
S4	MitB forges QR with intended details	✗	✓	✓	✓
S5	Tamper each field after code issued	✗	✓	✓	✓
S6	Replay code on old and new transaction	✓	✓	✓	✓
S7	Confirm after expiry	✓	✓	✓	✓
S8	50 concurrent submissions of one code	✓	✓	✓	✓
S9	3 wrong guesses, then correct code	✓	✓	✓	✓
S10	Server DB leak, forge transfer	✗	✗	✓	✓
S11	Third party can verify evidence	✗	✗	✓	✓*
S12	Non-canonical QR payloads	–	✓	✓	✓
S13	Malformed signatures	–	–	✓	✓

* In `sig_blind` the signature only proves that the device signed, not that the user saw the transaction.

display the attentive user has nothing to check, so the swaps succeed. S2b (the careless user) is ✗ everywhere: no cryptography saves a user who approves blindly.

Reading S2a, S3 and G7 (upper bound). The ✓ cells of S2a and S3, and the ✓ for G7, assume the *attentive* user: they say the mechanism *lets* the user detect the swap, not that a real user *will*. Real users lie between the attentive and careless models (section 2); S2b is the lower bound. We do not infer a detection rate from our own tests.

6.4 Online guessing

Scenario S9 confirms that three wrong guesses lock a transaction. To quantify the residual chance, `evaluation/bruteforce.py` simulates 100,000 transactions with three guesses each, for 3- and 4-digit codes, and compares the empirical per-transaction success probability with theory ($3/10^d$ for `hmac`; $1 - \binom{10^d-3}{3}/\binom{10^d}{3}$ for `baseline`'s three-code window), then extrapolates to 6 and 8 digits (table 7, fig. 6). Every 95% confidence interval contains the theoretical value.

Table 7: Per-transaction guessing probability (T, from `results/bruteforce.csv`). 6- and 8-digit rows are theoretical.

Scheme	Digits	Trials	Empirical [95% CI]	Theory
`hmac`	3	100,000	0.00279 [0.00248, 0.00314]	3.0×10^{-3}
`hmac`	4	100,000	0.00030 [0.00021, 0.00043]	3.0×10^{-4}
`hmac`	6	–	–	3.0×10^{-6}
`hmac`	8	–	–	3.0×10^{-8}
`baseline`	3	100,000	0.00886 [0.00830, 0.00946]	9.0×10^{-3}
`baseline`	4	100,000	0.00085 [0.00069, 0.00105]	9.0×10^{-4}
`baseline`	6	–	–	9.0×10^{-6}
`baseline`	8	–	–	9.0×10^{-8}

Limitation of G4 (per-transaction only). The three-attempt limit is per transaction; there is no per-user counter and transaction creation is not rate-limited (section 3). An attacker with the password but no phone can create k transactions and guess three times each, so the cumulative success probability is about $k \cdot p$. At the deployed digit counts this means roughly

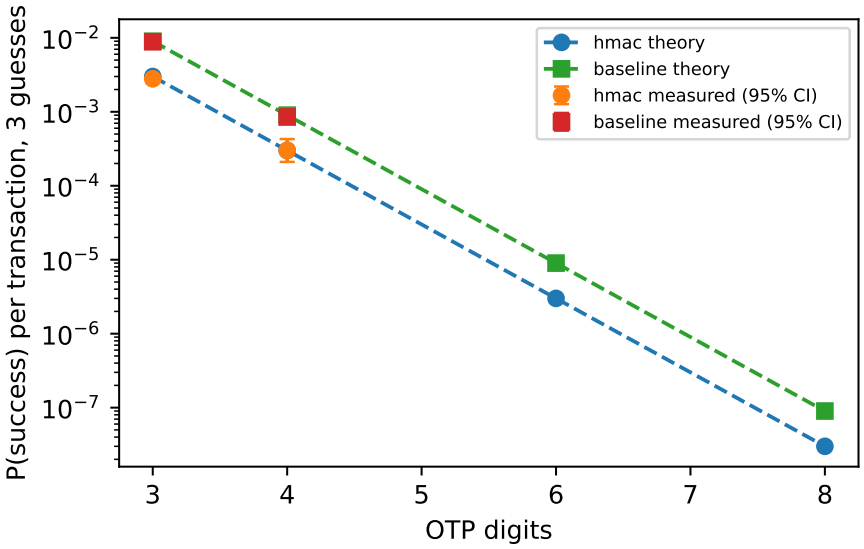


Figure 6: Empirical vs theoretical guessing probability (F5).

7.7×10^4 transactions for a 50% chance against `baseline` (6 digits, $p \approx 9.0 \times 10^{-6}$) and about 2.3×10^7 against `hmac` (8 digits, $p = 3.0 \times 10^{-8}$); `sig`/`sig_blind` cannot be guessed at all, as the attacker would have to forge an ECDSA signature. The ✓ for G4 therefore means only that the per-transaction lockout works. A per-user or per-device failure counter and a rate limit on transaction creation would close the gap, at the cost of letting an attacker who knows the password lock the victim out on purpose (a denial of service).

6.5 Server compromise and verifiable evidence

Scenarios S10 and S11 separate `hmac` from `sig`. In S10 the attacker holds a copy of the database and tries to forge a transfer. Because the server stores K_{totp} and K_{hmac} in the clear (deliberately, to make this scenario real), an attacker with the database can compute valid `baseline` and `hmac` codes — so both are ✗. The signing key, by contrast, never leaves the phone, so the leaked database contains only public keys and `sig`/`sig_blind` remain ✓. A real system would protect the symmetric keys in an HSM; we did not, in order to demonstrate exactly this difference.

S11 asks whether a third party can verify the stored evidence. For `baseline` and `hmac` the evidence is a symmetric OTP, checkable only by whoever holds the key, so the answer is *no*. For `sig` the evidence is a signature any party can verify with the public key, so the answer is *yes*. For `sig_blind` the answer is *yes* with the asterisk: the signature proves the device signed, not that the user saw the transaction. As set out in section 3, this is *verifiable evidence*, not non-repudiation: the authenticator is served by the bank, and the public key is attested by no independent party.

6.6 Summary against security goals

Table 8 collapses the scenario matrix onto the seven goals (generated by `make scenarios`).

7 Performance

All micro-benchmarks were produced by `make bench` and `make sizes` on the machine recorded in `results/bench_env.json`: an `arm64` macOS host, Python 3.15.2, `cryptography` 50.0.2 over OpenSSL 4.0.3.

Table 8: Security goals met by each mode (T6). Source: derived from `results/scenarios.csv`.

Goal	Description	baseline	hmac	sig	sig_blind
G1	Authorization integrity (code bound to the exact transaction)	✗	✓	✓	✓
G2	Freshness (no replay, no late use)	✓	✓	✓	✓
G3	Single use under concurrency	✓	✓	✓	✓
G4	Guessing limit per transaction	✓	✓	✓	✓
G5	Resilience to server database leak	✗	✗	✓	✓
G6	Verifiable evidence (third party can check)	✗	✗	✓	✓*
G7	What you see is what you sign (attentive user)	✗	✓	✓	✗

* In `sig_blind` the signature only proves that the device signed, not that the user saw the transaction.

7.1 P1: operation latency on the host

Table 9 gives the median and 95th-percentile time of each operation over 10,000 runs (`results/bench.csv`). Every operation is cheap. ECDSA P-256 signing is about 19 μs and verification about 49 μs; the HMAC code is about 5 μs. Ed25519 is shown for comparison only (it is not used by any mode): it signs more slowly but with similarly small cost.

Table 9: Operation latency on the host (P1, T7). Source: `results/bench.csv`.

Operation	Median (μs)	p95 (μs)
`canonical_encode`	3.50	3.71
`totp`	1.62	1.75
`hmac_otp`	5.25	5.42
`hmac_otp_verify`	5.42	5.58
`ecdsa_p256_sign`	19.17	19.96
`ecdsa_p256_verify`	48.50	51.04
`ed25519_sign`	26.67	27.79
`ed25519_verify`	67.96	76.29

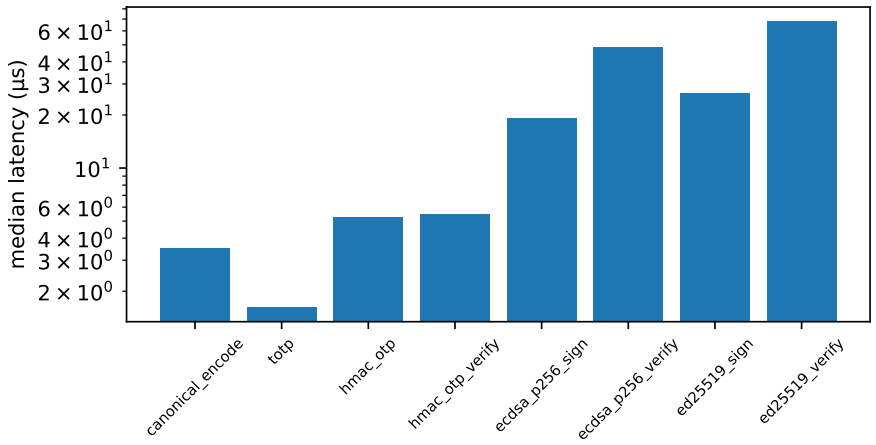


Figure 7: Operation latency on the host (F6).

7.2 P2: signing on a real phone

Table 10 reports the signing time measured in the authenticator itself on two phones (200 signatures each, `results/phone_bench.csv`). Safari on iOS coarsens `performance.now()` to

1 ms, so the per-sample median and p95 read as 0 and 1 ms and are not meaningful; the reliable figure is the mean over the whole run, which is well under a millisecond on both devices. Signing cost is therefore negligible next to the time the user spends scanning the QR and reading the screen.

Table 10: ECDSA P-256 signing on real phones (P2, T8). Source: `results/phone_bench.csv`. Median/p95 are limited by Safari's 1 ms clock.

Device	Browser	n	Mean (ms)	Median (ms)	p95 (ms)
iPhone Air	Safari 26.5	200	0.290	0.00	1.00
iPhone 13	Safari 26.4	200	0.415	0.00	1.00

7.3 P3: data sizes

Table 11 lists the sizes that matter for the QR channel (`results/sizes.csv`). A raw ECDSA P-256 signature is 64 bytes (the same as Ed25519), smaller than DER's 69–72 bytes, which is why the wire format is raw $r\|s$. The canonical transaction is 199 bytes, giving a 271-character QR payload at a version-12 symbol (error level M) — well within what a phone camera reads.

Table 11: Data sizes (P3). Source: `results/sizes.csv`.

Item	Value
TOTP code (digits)	6
Transaction OTP (digits)	8
ECDSA P-256 signature, raw (bytes)	64
ECDSA P-256 signature, DER min/max (bytes)	69 / 72
Ed25519 signature (bytes)	64
Canonical transaction (bytes)	199
QR payload (characters)	271
QR version (error level M)	12

On usability. A planned timing study (P4: completion time over six participants) was dropped from scope. The report therefore makes no usability-timing claim; the usability discussion is confined to the qualitative cost of scanning and reading, and to the attentive-user limitation already stated.

8 Discussion and Limitations

8.1 What each layer buys

Reading the matrix (tables 6 and 8) top to bottom gives a clear ordering. Transaction binding is the decisive step for RQ1: it turns the recipient and amount swaps from ✗ to ✓ (S2a, S3, S4, S5), which a plain OTP cannot do. Moving from a symmetric code to an asymmetric signature (RQ2) adds two properties: resilience to a database leak (S10) and third-party–verifiable evidence (S11), because the secret never leaves the phone. The trusted display (RQ3) is independent of both: `sig` and `sig_blind` share identical cryptography, yet `sig_blind` fails S2a and S3 — the strongest cryptography is wasted if the user is shown nothing to check. The careless-user column (S2b) is the sobering bound: no mechanism in this study defends a user who approves without looking.

8.2 Limitations

Attentive-user model. The attentive user is idealised: always reads, always notices. Real users sit between this and the careless model, and the literature shows a meaningful share of tampered transactions are approved, especially when only a digit or two of the account changes [8, 9]. The ✓ cells of S2a, S3 and G7 are therefore an *upper bound* on security; we did not run a user study, and we do not infer a detection rate from our own trials.

Guessing limit is per transaction (G4). As quantified in section 6, the lockout is per transaction with no per-user counter and no cap on transaction creation, so k transactions give about $k \cdot p$ cumulative success. This is weak for `baseline` and is closed only by a per-user/per-device counter and a creation rate limit, which introduce a denial-of-service trade-off.

Verifiable evidence is not non-repudiation (G6). The signature is checkable by a third party, but because the authenticator is served by the bank and the public key is attested by no independent party, the bank is not cryptographically bound by it. We claim only verifiable evidence.

Symmetric keys in the clear (G5). K_{totp} and K_{hmac} are stored in plaintext to make S10 concrete. A real deployment must hold them in an HSM; this is why `hmac` is ✗ for S10/S11 here.

Trusted, separate phone (A6). The whole scheme rests on the phone being trusted and separate from the PC. This is fragile: PC–phone cross-infection [10], cross-platform synchronisation [11], and running the bank app and the authenticator on one phone [12] all erase the gap. A6 (phone malware) is out of scope, and the authenticator being a web page served by the bank is itself a trust assumption worth stating.

Enrolment and other out-of-scope items. Enrolment assumes a clean, trust-on-first-use environment, and re-enrolment in the demo requires wiping the database; a real system needs an out-of-band recovery process. Push notifications, native apps, WebAuthn/SPC, multiple devices per user, and CSRF defences beyond `SameSite=Lax` are out of scope.

8.3 Positioning

None of the four mechanisms is novel (section 2); transaction binding and WYSIWYS are long established and, in the EU, mandated as dynamic linking [4]. The value of this work is the controlled comparison on one code base and the blind-signing experiment that isolates the display. Regulatory context for Vietnam (Decision 2345/QĐ-NHNN and any instrument that supersedes it) should be cited only after checking the currently effective text.

9 Conclusion

We built a demo bank with four interchangeable transaction-authentication modes and evaluated them under one threat model with an automated harness, a live man-in-the-browser demonstration, an online-guessing experiment, and micro-benchmarks, with every number produced by `make eval`.

The measurements answer the research questions. **RQ1:** binding the code to the transaction stops the recipient and amount tampering that a plain OTP lets through — `hmac` and `sig` block S2a, S3, S4 and S5, which `baseline` does not — and, by the A1/A2 equivalence, this covers real-time phishing as well. **RQ2:** asymmetric signatures add what a symmetric code cannot — survival of a database leak (S10) and third-party–verifiable evidence (S11) — because the key

never leaves the phone, at a signing cost of about 19 μs on the host and under a millisecond on a phone. **RQ3:** the trusted display is a separate and necessary layer: `sig_blind`, with identical cryptography but no display, fails exactly the attacks an attentive user would otherwise catch.

The honest reading is equally important: the guessing limit is only per transaction, "verifiable evidence" is not non-repudiation, the symmetric keys would need an HSM, and the attentive-user result is an upper bound. The strongest cryptography does not protect a user who is shown nothing, or who does not look.

Future work follows the limitations: a per-user guessing counter with a rate-limited creation path, an HSM for the symmetric keys, an out-of-band enrolment and recovery flow, independent attestation of the signing key to move from verifiable evidence toward non-repudiation, and a proper user study to replace the idealised attentive-user model with measured detection rates.

References

[1] A. Hiltgen, T. Kramp, and T. Weigold, "Secure internet banking authentication," *IEEE Security & Privacy*, vol. 4, no. 2, pp. 21–29, 2006.

[2] T. Weigold, T. Kramp, R. Hermann, F. Höring, P. Buhler, and M. Baentsch, "The Zurich trusted information channel — an efficient defence against man-in-the-middle and malicious software attacks," in *Trusted Computing (TRUST)*, 2008.

[3] D. M'Raihi, J. Rydell, S. Bajaj, S. Machani, and D. Naccache, "OCRA: OATH challenge-response algorithm," RFC 6287, 2011.

[4] European Commission, "Commission delegated regulation (eu) 2018/389 — regulatory technical standards for strong customer authentication (article 5, dynamic linking)," 2018, verify article text against the Official Journal.

[5] W3C, "Secure payment confirmation," W3C Working Draft, 2026, record the exact status (Candidate Recommendation Draft) when cited.

[6] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-based one-time password algorithm," RFC 4226, 2005.

[7] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," RFC 6238, 2011.

[8] M. AlZomai, B. AlFayyadh, A. Jøsang, and A. McCullagh, "An experimental investigation of the usability of transaction authorization in online bank security systems," in *Australasian Information Security Conference (AISC)*, 2008, verify venue, year and detection-rate figures against the original.

[9] V. Haupert and S. Gabert, "Where to look for what you see is what you sign? user confusion in transaction security," in *European Symposium on Research in Computer Security (ESORICS)*, 2019, verify pages and survey figures against the original.

[10] A. Dmitrienko, C. Liebchen, C. Rossow, and A.-R. Sadeghi, "On the (in)security of mobile two-factor authentication," in *Financial Cryptography and Data Security (FC)*, 2014.

[11] R. K. Konoth, V. van der Veen, and H. Bos, "How anywhere computing just killed your phone-based two-factor authentication," in *Financial Cryptography and Data Security (FC)*, 2016.

[12] V. Haupert and T. Müller, "On app-based matrix code authentication in online banking," in *International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

A Reproducing the results

The repository is a single project. With Python 3.11+ and Node 20+:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
make test        # Python unit/integration tests
make test-js     # authenticator crypto vs shared/vectors.json
make eval        # scenarios + brute-force + benchmarks -> results/
```

Every table and figure in this report maps to a file under `results/`:

Report element	Source file
Table 6 (T5)	`results/scenarios.csv` / `scenarios.tex`
Table 8 (T6)	derived from `results/scenarios.csv` / `goals.tex`
Table 7, fig. 6	`results/bruteforce.csv` / `bruteforce.pdf`
Table 9, fig. 7	`results/bench.csv` / `bench.pdf`
Table 10 (T8)	`results/phone_bench.csv`
Table 11	`results/sizes.csv`
Figure 5 (F4)	`results/screens/*.png`
Host environment	`results/bench_env.json`

B Live man-in-the-browser demo

The screenshots in fig. 5 were produced by running the attacker addon in a proxy between a compromised browser and the server:

```
pip install -e ".[attack]"            # mitmproxy into the venv (has segno)
make run MODE=sig                     # terminal 1
.venv/bin/mitmdump -s attacker/mitb_addon.py \
    --listen-port 8080 --ssl-insecure # terminal 2
# victim browser: proxy 127.0.0.1:8080, trust the mitmproxy CA (http://mitm.it)
# phone: connects to the server directly (not through the proxy)
```

Setting `MITB_FORGE_QR=1` also forges the QR to show the intended recipient (scenario S4); the phone is then deceived, but the signature no longer matches the transaction the server holds, so the bank rejects it.

C Demo accounts

Username	Account	Name	Balance (VND)
alice	0011223344	NGUYEN VAN A	100,000,000
bob	0123456789	TRAN THI B	5,000,000
mallory	0999888777	LE VAN M	0

Passwords are hashed with scrypt ($n = 2^{14}, r = 8, p = 1$). The symmetric keys are stored in plaintext *on purpose*, to make scenario S10 real; a production system must use an HSM.

D Contribution table

Member	Contribution
MEMBER 1	...
MEMBER 2	...
MEMBER 3	...
MEMBER 4	...
MEMBER 5	...
MEMBER 6	...